

A Certified Positivity Theorem for OEIS A348369

Classical proof and Lean formalization in parallel

Thomas Scheuerle

September 22, 2026

Abstract

Let $\mathcal{A}$ be the increasing sequence OEIS A328596, whose elements are the positive integers whose reversed binary expansions are Lyndon words. OEIS A348369 counts, for the n th element of $\mathcal{A}$, its unordered representations as a sum of two earlier elements of $\mathcal{A}$. We prove that every element of $\mathcal{A}$ other than 1 has such a representation. Consequently, $a(n) \geq 1$ for every $n > 1$ in A348369. The proof removes the least-significant set bit. Each mathematical definition and argument below is immediately followed by the corresponding Lean 4 fragment, so the ordinary proof and its machine-checked realization can be read together.

1 The two sequences and the theorem

Definition 1.1 (Reversed binary word). For $N \geq 0$, let $\text{revbin}(N)$ be the binary expansion of N written from least significant bit to most significant bit, represented as a word over $0 < 1$. Thus $\text{revbin}(26) = 01011$.

Lean realization.

```
abbrev Bit := Bool
def revBinary (n : Nat) : List Bit := n.bits
example : revBinary 26 =
  [false, true, false, true, true] := rfl
```

Definition 1.2 (Lyndon word). A nonempty word w is Lyndon when it is lexicographically smaller than every nontrivial cyclic rotation of itself. Equivalently for the words used here, w is smaller than every proper nonempty suffix.

Lean realization.

```
def IsLyndon {α : Type*} [LinearOrder α] (w : List α) : Prop :=
  w ≠ [] ∧
  ∀ i, 0 < i → i < w.length → w < w.rotate i

/-- An auxiliary formulation that compares a word directly with each proper
nonempty suffix. For the binary lemmas below, this is often easier to use than
the rotation-based definition. -/
```

Definition 1.3 (A328596). Put

$$\mathcal{A} = \{N \in \mathbb{N} : N > 0 \text{ and } \text{revbin}(N) \text{ is Lyndon}\}.$$

Write its increasing enumeration as $A(1) < A(2) < \dots$.

Lean realization.

```
def InA328596 (n : Nat) : Prop :=
  0 < n /\ IsLyndon (revBinary n)
```

Definition 1.4 (A348369). For $n \geq 1$, let $a(n)$ be the number of unordered pairs $\{k, m\}$ with $0 < k, m < n$ and

$$A(n) = A(k) + A(m).$$

This is OEIS A348369. Its first term is $a(1) = 0$.

Lean realization.

The supplied Lean development formalizes the existence theorem on the values $A(n)$ rather than the finite counting function $a(n)$. The final corollary below explains why this is exactly what is needed for positivity of the count.

Theorem 1.5 (Main theorem). *For every $n > 1$, one has $a(n) \geq 1$.*

2 Binary structure

Lemma 2.1 (Powers of two). *For every $k \geq 0$,*

$$\text{revbin}(2^k) = 0^k 1,$$

and $2^k \in \mathcal{A}$.

Proof. Multiplication by 2 prepends a zero to the least-significant-bit-first word. Starting from $\text{revbin}(1) = 1$ gives the displayed form. The word $0^k 1$ is smaller than each of its proper nonempty suffixes, hence is Lyndon. $\square$

Lean realization.

```
theorem revBinary_pow_two (k : Nat) :
  revBinary (2 ^ k) = List.replicate k false ++ [true] := by
  induction k with
  | zero =>
    simp [revBinary_one]
  | succ k ih =>
    have hk : 2 ^ k ≠ 0 := by
      have hk' : 0 < 2 ^ k := by
        positivity
      exact Nat.ne_of_gt hk'
    calc
    revBinary (2 ^ (k + 1)) = revBinary (2 * (2 ^ k)) := by
      rw [Nat.pow_succ, Nat.mul_comm]
    _ = false :: revBinary (2 ^ k) := by
      simp [revBinary_two_mul hk]
    _ = false :: (List.replicate k false ++ [true]) := by
      rw [ih]
    _ = List.replicate (k + 1) false ++ [true] := by
      simp [List.replicate]

/-- Multiplying a positive number by '2^k' prefixes 'k' zero bits in the
reversed binary expansion. -/
```

Lemma 2.2 (Odd-factor shape). *If $N = 2^k(2r + 1)$, then*

$$\text{revbin}(N) = 0^k 1 \text{ revbin}(r).$$

If $r > 0$ and the least-significant set bit 2^k is removed, then

$$\text{revbin}(N - 2^k) = 0^{k+1} \text{ revbin}(r).$$

Proof. The factor 2^k contributes k initial zeros. Since $2r + 1$ is odd, its first reversed bit is 1 and the remaining bits are those of r . Moreover $N - 2^k = 2^{k+1}r$, which gives the second identity. $\square$

Lean realization.

```

theorem revBinary_pow_two_mul_odd (k r : Nat) :
  revBinary (2 ^ k * (2 * r + 1)) = List.replicate k false ++ (true ::
    revBinary r) := by
  calc
    revBinary (2 ^ k * (2 * r + 1)) = List.replicate k false ++ revBinary (2 * r +
      1) := by
      exact revBinary_mul_pow_two k (2 * r + 1) (by omega)
    _ = List.replicate k false ++ (true :: revBinary r) := by
      simp [revBinary_two_mul_add_one]

/-- Clearing the least-significant set bit of ' $2^k * (2r + 1)$ ' for ' $r > 0$ '
adds one more leading '0' in reversed binary. -/

```

3 The Lyndon transformation

Lemma 3.1 (Zero-block step). *Let u be a binary word. If $0^k 1 u 1$ is Lyndon, then $0^{k+1} u 1$ is Lyndon.*

Proof. Use the suffix characterization. First delete the distinguished 1 after the initial zero block. For a suffix beginning inside the zero block, lexicographic comparison is determined after cancelling the common zeros; the additional nonempty block of zeros is smaller than the terminal word $u 1$. For a suffix beginning to the right of the distinguished 1, the required inequality follows transitively from the corresponding suffix inequality for $0^k 1 u 1$. The resulting word remains suffix-Lyndon. Prefixing one additional 0 preserves that property because every word considered ends in 1. The equivalence of suffix and rotation formulations completes the proof. $\square$

Lean realization.

```

theorem remove_first_true_preserves_isSuffixLyndon (k : Nat) (u : List Bool)
  (h : IsSuffixLyndon (List.replicate k false ++ (true :: (u ++ [true])))) :
  IsSuffixLyndon (List.replicate k false ++ (u ++ [true])) := by
  let t := u ++ [true]
  have hyx : List.replicate k false ++ t < List.replicate k false ++ (true :: t)
    := by
    simp [t] using append_left_lt_of_lt (List.replicate k false) (
      append_true_lt_true_cons u)
  constructor
  · simp [t]
  · intro i hi0 hi

```

```

by_cases hik : i ≤ k
· have hdecomp : k = (k - i) + i := by omega
  have hy : List.replicate k false ++ t =
    List.replicate (k - i) false ++ (List.replicate i false ++ t) := by
    rw [hdecomp, List.replicate_add, List.append_assoc]
    simp
  have hdropy : (List.replicate k false ++ t).drop i = List.replicate (k - i)
false ++ t := by
  rw [List.drop_append_of_le_length]
  · simp [hik, t]
  · simpa using hik
calc
  List.replicate k false ++ t
    = List.replicate (k - i) false ++ (List.replicate i false ++ t) := hy
  _ < List.replicate (k - i) false ++ t :=
    append_left_lt_of_lt (List.replicate (k - i) false)
      (replicate_falses_lt_self_append_true_of_pos hi0 u)
  _ = (List.replicate k false ++ t).drop i := hdropy.symm
· have hklt : k < i := lt_of_not_ge hik
  let j := i - k
  have hjpos : 0 < j := by
    dsimp [j]
    omega
  have hjlen : j < t.length := by
    dsimp [j] at *
    simp [t] at hi ⊢
    omega
  have hdropy : (List.replicate k false ++ t).drop i = t.drop j := by
    calc
      (List.replicate k false ++ t).drop i
        = ((List.replicate k false ++ t).drop k).drop j := by
          rw [show i = k + j by
            dsimp [j]
            omega, List.drop_drop]
        _ = t.drop j := by
          rw [List.drop_append_of_le_length]
          · simp [t]
          · simp
  have hdropx : (List.replicate k false ++ (true :: t)).drop (i + 1) = t.drop
j := by
  calc
    (List.replicate k false ++ (true :: t)).drop (i + 1)
      = ((List.replicate k false ++ (true :: t)).drop (k + 1)).drop j :=
by
    rw [show i + 1 = (k + 1) + j by
      dsimp [j]
      omega, List.drop_drop]
    _ = t.drop j := by
      have hchk1 : (List.replicate k false ++ (true :: t)).drop (k + 1) = t
:= by
      calc
        (List.replicate k false ++ (true :: t)).drop (k + 1)
          = ((List.replicate k false ++ (true :: t)).drop k).drop 1 :=
by
          rw [show k + 1 = k + 1 by omega, List.drop_drop]
          _ = (true :: t).drop 1 := by
            rw [List.drop_append_of_le_length]

```

```

      · simp
      · simp
    _ = t := by simp
  rw [hxx1]
  have hxlt : List.replicate k false ++ (true :: t) < t.drop j := by
    have hxlen : i + 1 < (List.replicate k false ++ (true :: t)).length := by
      simp [t] at hi ⊢
    omega
  have htmp := h.lt_drop (Nat.succ_pos _) hxlen
  rw [hdropx] at htmp
  exact htmp
  rw [hdropy]
  exact lt_trans hyx hxlt

/-- Suffix-form version of the main combinatorial transformation:
from '0^k 1 u 1', delete the first '1' after the initial zero block, then
prepend one more '0'. -/

```

Proposition 3.2 (Closure under clearing the least-significant set bit). *Let $r > 0$. If $N = 2^k(2r + 1)$ belongs to $\mathcal{A}$, then $N - 2^k = 2^{k+1}r$ also belongs to $\mathcal{A}$.*

Proof. Because $r > 0$, its reversed binary word ends in 1; write $\text{revbin}(r) = u1$. By Lemma 2.2, $\text{revbin}(N) = 0^k 1 u 1$. This word is Lyndon by $N \in \mathcal{A}$. Lemma 3.1 shows that $0^{k+1}u1$ is Lyndon. Lemma 2.2 identifies it with $\text{revbin}(N - 2^k)$, and the latter number is positive. $\square$

Lean realization.

```

theorem clear_lsb_shape_mem_a328596 (k r : Nat) (hr : r ≠ 0)
  (hA : InA328596 (2 ^ k * (2 * r + 1))) :
  InA328596 (2 ^ (k + 1) * r) := by
  constructor
  · exact Nat.mul_pos (by positivity) (Nat.pos_of_ne_zero hr)
  · rcases revBinary_pos_ends_with_true (Nat.pos_of_ne_zero hr) with ⟨u, hu⟩
    have hword : IsSuffixLyndon (List.replicate k false ++ (true :: (u ++ [true])))
    := by
      have hly : IsLyndon (List.replicate k false ++ (true :: (u ++ [true]))) :=
      by
        simp [revBinary_pow_two_mul_odd, hu] using hA.2
      exact hly.toIsSuffixLyndon
    have hsmall : IsLyndon (List.replicate (k + 1) false ++ (u ++ [true])) :=
      zero_block_step_isLyndon k u hword
    simp [revBinary_mul_pow_two (k + 1) r hr, hu, List.append_assoc] using
    hsmall

```

4 Additive decomposition and positivity

Theorem 4.1 (Additive decomposition). *If $N > 1$ and $N \in \mathcal{A}$, then there exist $X, Y \in \mathcal{A}$ such that $N = X + Y$.*

Proof. Write $N = 2^k q$, where q is odd.

If $q = 1$, then $N = 2^k$. Since $N > 1$, $k = j + 1$ for some j , and

$$N = 2^j + 2^j.$$

Both summands belong to $\mathcal{A}$ by Lemma 2.1.

Now suppose $q > 1$. Write $q = 2r + 1$. Then $r > 0$. The least-significant set bit $X = 2^k$ belongs to $\mathcal{A}$ by Lemma 2.1. Proposition 3.2 shows that $Y = 2^{k+1}r = N - 2^k$ also belongs to $\mathcal{A}$. Hence

$$N = 2^k + 2^{k+1}r = X + Y,$$

which proves the assertion. $\square$

Lean realization.

```

theorem a328596_additive {n : Nat} (hn : 1 < n) (hA : InA328596 n) :
  ∃ a b, InA328596 a ∧ InA328596 b ∧ n = a + b := by
  let k := padicValNat 2 n
  let q := n.divMaxPow 2
  have hfact : 2 ^ k * q = n := by
    simp [k, q] using (pow_padicValNat_mul_divMaxPow 2 n)
  by_cases hq1 : q = 1
  · have hkshape : n = 2 ^ k := by
      simp [hq1] using hfact.symm
    rcases Nat.eq_zero_or_pos k with hk0 | hkpos
    · have hnone : n = 1 := by simp [hk0] using hkshape
      omega
    · obtain ⟨j, hj⟩ := Nat.exists_eq_succ_of_ne_zero (Nat.ne_of_gt hkpos)
      refine ⟨2 ^ j, 2 ^ j, two_pow_mem_a328596 j, two_pow_mem_a328596 j, ?_⟩
      calc
        n = 2 ^ (j + 1) := by simp [hj] using hkshape
        _ = 2 ^ j + 2 ^ j := by
          rw [Nat.pow_succ]
          nth_rw 2 [show 2 = 1 + 1 by rfl]
          rw [Nat.mul_add]
          simp
  · have hqpos : 0 < q := by
      apply Nat.pos_of_ne_zero
      intro hq0
      rw [hq0, mul_zero] at hfact
      exact (Nat.ne_of_gt hA.1) hfact.symm
  have hqodd : ¬ 2 ∣ q := by
    simp [q] using (Nat.not_dvd_divMaxPow (by decide : 1 < 2) (Nat.ne_of_gt hA
      .1))
  have hbodd : q.bodd = true := by
    cases hqbod : q.bodd <=> simp [Nat.mod_two_of_bodd, hqbod, Nat.
      dvd_iff_mod_eq_zero] at hqodd ⊢
  let r := q.div2
  have hqshape : q = 2 * r + 1 := by
    have hbit : Nat.bit true r = q := by
      simp [r, hbodd] using (Nat.bit_bodd_div2 q)
    simp [Nat.bit, r] using hbit.symm
  have hr : r ≠ 0 := by
    intro hr0
    have : q = 1 := by simp [hqshape, r, hr0]
    exact hq1 this
  have hshape : n = 2 ^ k * (2 * r + 1) := by
    rw [← hfact, hqshape]
  have hsmall : InA328596 (2 ^ (k + 1) * r) := by
    have hAshape : InA328596 (2 ^ k * (2 * r + 1)) := by
      simp [hshape] using hA
    exact clear_lsb_shape_mem_a328596 k r hr hAshape

```

```

refine ⟨2 ^ k, 2 ^ (k + 1) * r, two_pow_mem_a328596 k, hsmall, ?_⟩
calc
  n = 2 ^ k * (2 * r + 1) := hshape
  _ = 2 ^ k + 2 ^ (k + 1) * r := by
    rw [Nat.mul_add, Nat.mul_one, Nat.pow_succ]
    ac_rfl

```

Proof of Theorem 1.5. Fix $n > 1$ and put $N = A(n)$. Since the increasing sequence A begins with $A(1) = 1$, we have $N > 1$. Theorem 4.1 supplies $X, Y \in \mathcal{A}$ with $N = X + Y$. Positivity gives $0 < X, Y < N$. Hence there are indices $0 < k, m < n$ with $X = A(k)$ and $Y = A(m)$. Thus the unordered pair $\{k, m\}$ is counted by $a(n)$, and therefore $a(n) \geq 1$. $\square$

Lean realization.

The machine-checked mathematical core is exactly `a328596_additive`. A fully formal theorem mentioning `a(n)` additionally requires Lean definitions of the increasing enumeration of `InA328596` and of the finite quotient count of unordered pairs. Those bookkeeping definitions are not presented here.

Remark 4.2 (Strength of the result). The proof constructs a witness. For non-powers of two it selects the least-significant set bit $2^{v_2(N)}$ and the complementary summand. Accordingly, it proves existence rather than merely deriving positivity from computed terms.

Lean realization.

The witness is exposed by the two branches of `a328596_additive`: either $(2^j, 2^j)$ or $(2^k, 2^{k+1}r)$.

Reproducibility note

The formal development consists of `BinaryWords.lean`, `Lyndon.lean`, and `Sequence.lean`, using Lean 4 and Mathlib. Every nontrivial Lean declaration cited in the article is reproduced in full below. For complete reproducibility, Appendix A also contains the verbatim contents of all three Lean source files; no proof is abbreviated by an ellipsis.

A Complete Lean 4 source files

This appendix reproduces the complete machine-checked development verbatim, including imports, namespaces, definitions, auxiliary lemmas, sanity checks, and the final theorem. The three modules are shown in dependency order.

A.1 BinaryWords.lean

```
import Mathlib.Data.Nat.Bits
import Mathlib.Tactic

/-!
# Reversed Binary Words

This file defines the reversed binary expansion used in the OEIS A328596
formalization and proves a first structural lemma for powers of two.
-/

namespace A328596

/-- We represent binary words as lists of bits.
'false' means '0' and 'true' means '1'. -/
abbrev Bit := Bool

/-- 'revBinary n' is the ordinary binary expansion of 'n', written from
least significant bit to most significant bit.

For example, '26 = 110102', so 'revBinary 26 = [0, 1, 0, 1, 1]'. -/
def revBinary (n : Nat) : List Bit :=
  n.bits

@[simp] theorem revBinary_zero : revBinary 0 = [] := by
  rfl

@[simp] theorem revBinary_one : revBinary 1 = [true] := by
  simp [revBinary]

@[simp] theorem revBinary_two_mul {n : Nat} (hn : n ≠ 0) :
  revBinary (2 * n) = false :: revBinary n := by
  simp [revBinary, Nat.bit0_bits, hn]

@[simp] theorem revBinary_two_mul_add_one (n : Nat) :
  revBinary (2 * n + 1) = true :: revBinary n := by
  simp [revBinary]

/-- The reversed binary word of '2k' is '0k 1'. -/
theorem revBinary_pow_two (k : Nat) :
  revBinary (2 ^ k) = List.replicate k false ++ [true] := by
  induction k with
  | zero =>
    simp [revBinary_one]
  | succ k ih =>
    have hk : 2 ^ k ≠ 0 := by
      have hk' : 0 < 2 ^ k := by
        positivity
      exact Nat.ne_of_gt hk'
```

```

    calc
      revBinary (2 ^ (k + 1)) = revBinary (2 * (2 ^ k)) := by
        rw [Nat.pow_succ, Nat.mul_comm]
      _ = false :: revBinary (2 ^ k) := by
        simp [revBinary_two_mul hk]
      _ = false :: (List.replicate k false ++ [true]) := by
        rw [ih]
      _ = List.replicate (k + 1) false ++ [true] := by
        simp [List.replicate]

/-- Multiplying a positive number by '2^k' prefixes 'k' zero bits in the
reversed binary expansion. -/
theorem revBinary_mul_pow_two (k n : Nat) (hn : n ≠ 0) :
  revBinary (2 ^ k * n) = List.replicate k false ++ revBinary n := by
  induction k with
  | zero =>
    simp
  | succ k ih =>
    have hk : 2 ^ k * n ≠ 0 := by
      exact Nat.mul_ne_zero (pow_ne_zero k (by decide)) hn
    have hmul : 2 ^ (k + 1) * n = 2 * (2 ^ k * n) := by
      rw [Nat.pow_succ]
    ac_rfl
    calc
      revBinary (2 ^ (k + 1) * n) = revBinary (2 * (2 ^ k * n)) := by rw [hmul]
      _ = false :: revBinary (2 ^ k * n) := by
        simp [revBinary_two_mul hk]
      _ = false :: (List.replicate k false ++ revBinary n) := by
        rw [ih]
      _ = List.replicate (k + 1) false ++ revBinary n := by
        simp [List.replicate]

/-- Splitting off the least-significant '1' from an odd factor exposes the
shape used in the additive decomposition argument. -/
theorem revBinary_pow_two_mul_odd (k r : Nat) :
  revBinary (2 ^ k * (2 * r + 1)) = List.replicate k false ++ (true ::
  revBinary r) := by
  calc
    revBinary (2 ^ k * (2 * r + 1)) = List.replicate k false ++ revBinary (2 * r +
    1) := by
      exact revBinary_mul_pow_two k (2 * r + 1) (by omega)
    _ = List.replicate k false ++ (true :: revBinary r) := by
      simp [revBinary_two_mul_add_one]

/-- Clearing the least-significant set bit of '2^k * (2r + 1)' for 'r > 0'
adds one more leading '0' in reversed binary. -/
theorem revBinary_clear_lsb (k r : Nat) (hr : r ≠ 0) :
  revBinary (2 ^ k * (2 * r + 1) - 2 ^ k) =
  List.replicate (k + 1) false ++ revBinary r := by
  have hsub : 2 ^ k * (2 * r + 1) - 2 ^ k = 2 ^ k * (2 * r) := by
    calc
      2 ^ k * (2 * r + 1) - 2 ^ k = 2 ^ k * (2 * r + 1) - 2 ^ k * 1 := by simp
      _ = 2 ^ k * ((2 * r + 1) - 1) := by rw [Nat.mul_sub_left_distrib]
      _ = 2 ^ k * (2 * r) := by simp
  have hmul : 2 ^ k * (2 * r) = 2 ^ (k + 1) * r := by
    rw [Nat.pow_succ]
  ac_rfl

```

```

calc
  revBinary (2 ^ k * (2 * r + 1) - 2 ^ k)
    = revBinary (2 ^ k * (2 * r)) := by rw [hsub]
  _ = revBinary (2 ^ (k + 1) * r) := by rw [hmul]
  _ = List.replicate (k + 1) false ++ revBinary r := by
    exact revBinary_mul_pow_two (k + 1) r hr

/-- Every positive reversed binary expansion ends with a '1'. -/
theorem revBinary_pos_ends_with_true {n : Nat} (hn : 0 < n) :
  ∃ u, revBinary n = u ++ [true] := by
  induction n using Nat.strong_induction_on with
  | h n ih =>
    have hn0 : n ≠ 0 := Nat.ne_of_gt hn
    by_cases hodd : n.bodd = true
    · cases hdiv : n.div2 with
      | zero =>
        use []
        have hn1 : n = 1 := by
          have hbit : Nat.bit true 0 = n := by
            simp [hodd, hdiv] using Nat.bit_bodd_div2 n
          simp [Nat.bit] using hbit.symm
        simp [hn1, revBinary_one]
      | succ m =>
        have hsmall : Nat.succ m < n := by
          simp [hdiv] using Nat.binaryRec_decreasing hn0
        rcases ih (Nat.succ m) hsmall (Nat.succ_pos _) with ⟨u, hu⟩
        use true :: u
        have hbit : Nat.bit true (Nat.succ m) = n := by
          simp [hodd, hdiv] using Nat.bit_bodd_div2 n
        have hnexpr : n = 2 * Nat.succ m + 1 := by
          simp [Nat.bit] using hbit.symm
        rw [hnexpr, revBinary_two_mul_add_one, hu]
        simp
    · have hodd0 : n.bodd = false := by
        cases hbod : n.bodd <=> simp_all
      cases hdiv : n.div2 with
      | zero =>
        have : False := by
          have hbit : Nat.bit false 0 = n := by
            simp [hodd0, hdiv] using Nat.bit_bodd_div2 n
          have hnzero : n = 0 := by
            simp [Nat.bit] using hbit.symm
          exact hn0 hnzero
        exact this.elim
      | succ m =>
        have hsmall : Nat.succ m < n := by
          simp [hdiv] using Nat.binaryRec_decreasing hn0
        rcases ih (Nat.succ m) hsmall (Nat.succ_pos _) with ⟨u, hu⟩
        use false :: u
        have hbit : Nat.bit false (Nat.succ m) = n := by
          simp [hodd0, hdiv] using Nat.bit_bodd_div2 n
        have hnexpr : n = 2 * Nat.succ m := by
          simp [Nat.bit] using hbit.symm
        rw [hnexpr, revBinary_two_mul (by simp [hdiv]), hu]
        simp

example : revBinary 26 = [false, true, false, true, true] := rfl

```

```
end A328596
```

A.2 Lyndon.lean

```
import Mathlib.Data.List.Lex
import Mathlib.Data.List.Infix
import Mathlib.Data.List.Rotate
import Mathlib.Tactic

/-!
# Lyndon Words

This file contains the transparent list-based definition of a Lyndon word used
throughout the project.
-/

namespace A328596

/-- A word is Lyndon if it is nonempty and is lexicographically strictly smaller
than every nontrivial cyclic rotation.

This is the direct list-based version of the mathematical definition. -/
def IsLyndon {α : Type*} [LinearOrder α] (w : List α) : Prop :=
  w ≠ [] ∧
  ∀ i, 0 < i → i < w.length → w < w.rotate i

/-- An auxiliary formulation that compares a word directly with each proper
nonempty suffix. For the binary lemmas below, this is often easier to use than
the rotation-based definition. -/
def IsSuffixLyndon {α : Type*} [LinearOrder α] (w : List α) : Prop :=
  w ≠ [] ∧
  ∀ i, 0 < i → i < w.length → w < w.drop i

/-- A Boolean checker for the Lyndon condition, used for small computational
sanity checks.

It checks the nonempty condition and then tests all rotations with cut positions
'i < w.length', treating 'i = 0' as the trivial rotation that we ignore. -/
def isLyndonBool {α : Type*} [LinearOrder α] (w : List α) : Bool :=
  decide (w ≠ []) &&
  (List.range w.length).all fun i => decide (i = 0 ∨ w < w.rotate i)

theorem IsLyndon.ne_nil {α : Type*} [LinearOrder α] {w : List α} (h : IsLyndon w)
  :
  w ≠ [] :=
  h.1

theorem IsLyndon.lt_rotate {α : Type*} [LinearOrder α] {w : List α} (h : IsLyndon
  w)
  {i : Nat} (hi0 : 0 < i) (hi : i < w.length) :
  w < w.rotate i :=
  h.2 i hi0 hi

theorem IsSuffixLyndon.ne_nil {α : Type*} [LinearOrder α] {w : List α}
```

```

(h : IsSuffixLyndon w) : w ≠ [] :=
h.1

theorem IsSuffixLyndon.lt_drop {α : Type*} [LinearOrder α] {w : List α}
  (h : IsSuffixLyndon w) {i : Nat} (hi0 : 0 < i) (hi : i < w.length) :
  w < w.drop i :=
h.2 i hi0 hi

theorem cons_lt_cons {α : Type*} [LinearOrder α] {a : α} {u v : List α}
  (h : u < v) : a :: u < a :: v := by
exact (List.lt_iff_lex_lt _ _).2 <| List.Lex.cons ((List.lt_iff_lex_lt _ _).1 h)

theorem append_right_lt_of_lt {α : Type*} [LinearOrder α] {u v : List α} (t :
  List α)
  (h : u < v) : u < v ++ t := by
exact (List.lt_iff_lex_lt _ _).2 <|
  List.Lex.append_right (r := (· < ·)) t ((List.lt_iff_lex_lt _ _).1 h)

theorem append_left_lt_of_lt {α : Type*} [LinearOrder α] {u v : List α} (p : List
  α)
  (h : u < v) : p ++ u < p ++ v := by
exact (List.lt_iff_lex_lt _ _).2 <|
  List.Lex.append_left (R := (· < ·)) ((List.lt_iff_lex_lt _ _).1 h) p

theorem append_left_cancel_lt {α : Type*} [LinearOrder α] {p u v : List α}
  (h : p ++ u < p ++ v) : u < v := by
induction p with
| nil => simp using h
| cons a p ih =>
  have h' : p ++ u < p ++ v := by
    exact (List.lex_cons_iff).1 h
  exact ih h'

theorem append_right_lt_of_lt_of_eq_length {α : Type*} [LinearOrder α]
  {u v : List α} (t : List α) (h : u < v) (hlen : u.length = v.length) :
  u ++ t < v ++ t := by
induction u generalizing v with
| nil =>
  cases v with
  | nil => cases lt_irrefl [] h
  | cons b v => simp at hlen
| cons a u ih =>
  cases v with
  | nil => simp at hlen
  | cons b v =>
    rcases lt_or_eq_of_le (List.head_le_of_lt h) with hab | rfl
    · exact (List.lt_iff_lex_lt _ _).2 <| List.Lex.rel hab
    · have htail : u < v := by simp [List.lex_cons_iff] using h
      have hlen' : u.length = v.length := by simp using hlen
      exact cons_lt_cons (ih htail hlen')

theorem append_lt_of_lt_of_not_prefix {α : Type*} [LinearOrder α]
  {u v t : List α} (h : u < v) (hprefix : ¬ u <+: v) : u ++ t < v := by
induction u generalizing v t with
| nil =>
  simp using hprefix

```

```

| cons a u ih =>
  cases v with
| nil => cases h
| cons b v =>
  rcases lt_or_eq_of_le (List.head_le_of_lt h) with hab | rfl
  · exact (List.lt_iff_lex_lt _ _).2 <| List.Lex.rel hab
  · have htail : u < v := by simpa [List.lex_cons_iff] using h
    have hprefix' : ¬ u <+: v := by
      intro hp
      exact hprefix (List.cons_prefix_cons.mpr ⟨rfl, hp⟩)
    exact cons_lt_cons (ih htail hprefix')

theorem IsSuffixLyndon.toIsLyndon {α : Type*} [LinearOrder α] {w : List α}
  (h : IsSuffixLyndon w) : IsLyndon w := by
  constructor
  · exact h.1
  · intro i hi0 hi
    rw [List.rotate_eq_drop_append_take hi.le]
    exact append_right_lt_of_lt (List.take i w) (h.lt_drop hi0 hi)

theorem IsLyndon.drop_not_prefix {α : Type*} [LinearOrder α] {w : List α}
  (h : IsLyndon w) {i : Nat} (hi0 : 0 < i) (hi : i < w.length) :
  ¬ w.drop i <+: w := by
  intro hprefix
  let v := w.drop i
  let u := w.take i
  let z := w.drop v.length
  have hsplit : w = u ++ v := by
    simpa [u, v] using (List.take_append_drop i w).symm
  have hrot_i : w < v ++ u := by
    have hrot := h.lt_rotate hi0 hi
    rw [List.rotate_eq_drop_append_take hi.le] at hrot
    simpa [u, v] using hrot
  have hprefix_split : w = v ++ w.drop v.length := by
    exact List.prefix_append_drop hprefix
  have hzltu : z < u := by
    rw [hprefix_split] at hrot_i
    simpa [z] using append_left_cancel_lt hrot_i
  have hvpos : 0 < v.length := by
    simp [v]
    omega
  have hvlt : v.length < w.length := by
    simp [v]
    omega
  have hzlen : z.length = u.length := by
    simp [u, v, z]
    omega
  have hrot_v : w < z ++ v := by
    have hrot : v ++ z < z ++ v := by
      have hrot' := h.lt_rotate hvpos hvlt
      rw [hprefix_split, List.rotate_append_length_eq] at hrot'
      simpa [z] using hrot'
    exact hprefix_split.symm ▸ hrot
  have hback : z ++ v < w := by
    have htmp : z ++ v < u ++ v :=
      append_right_lt_of_eq_length v hzltu hzlen
    exact hsplit.symm ▸ htmp

```

```

    exact (lt_asymm hrot_v hback).elim

theorem IsLyndon.toIsSuffixLyndon {α : Type*} [LinearOrder α] {w : List α}
  (h : IsLyndon w) : IsSuffixLyndon w := by
  constructor
  · exact h.1
  · intro i hi0 hi
    let v := w.drop i
    let u := w.take i
    have hrot : w < v ++ u := by
      have hrot' := h.lt_rotate hi0 hi
      rw [List.rotate_eq_drop_append_take hi.le] at hrot'
      simpa [u, v] using hrot'
    by_contra hnv
    have hvne : v ≠ w := by
      intro hEq
      have : v.length < w.length := by
        simp [v]
        omega
      simpa [hEq] using this
    have hvw : v < w := lt_of_le_of_ne (le_of_not_gt hnv) hvne
    by_cases hprefix : v <+: w
    · exact (h.drop_not_prefix hi0 hi) hprefix
    · have hcontra : v ++ u < w := append_lt_of_lt_of_not_prefix hvw hprefix
      exact (lt_asymm hrot hcontra).elim

theorem isLyndon_singleton {α : Type*} [LinearOrder α] (a : α) :
  IsLyndon [a] := by
  constructor
  · simp
  · intro i hi0 hi
    have hlt : i < 1 := by simpa using hi
    omega

theorem isSuffixLyndon_singleton {α : Type*} [LinearOrder α] (a : α) :
  IsSuffixLyndon [a] := by
  constructor
  · simp
  · intro i hi0 hi
    have hlt : i < 1 := by simpa using hi
    omega

/-- If a binary suffix-Lyndon word ends in '1', then prefixing one more '0'
produces another suffix-Lyndon word.

This is one of the structural closure properties behind the least-significant-
bit decomposition. -/
theorem prepend_false_preserves_isSuffixLyndon (u : List Bool)
  (h : IsSuffixLyndon (u ++ [true])) :
  IsSuffixLyndon (false :: (u ++ [true])) := by
  constructor
  · simp
  · intro i hi0 hi
    cases i with
    | zero => cases Nat.lt_asymm hi0 hi0
    | succ j =>
      have hj : j < (u ++ [true]).length := by

```

```

      simp using hi
    have hjle : j ≤ u.length := by
      simp using hj
    have hdrop : (u ++ [true]).drop j = u.drop j ++ [true] := by
      simp using List.drop_append_of_le_length (l₁ := u) (l₂ := [true]) hjle
    cases huj : u.drop j with
    | nil =>
      rw [show (false :: (u ++ [true])).drop (j + 1) = [true] by simp [
hdrop, huj]]
      exact (List.lt_iff_lex_lt _ _).2 <| List.Lex.rel (show false < true
by decide)
    | cons b t =>
      cases b with
    | false =>
      have hju : j < u.length := by
      have hlen' : 0 < (u.drop j).length := by
      simp [huj]
      rw [List.length_drop] at hlen'
      omega
      have htail : (u ++ [true]).drop (j + 1) = t ++ [true] := by
      rw [← List.drop_drop, hdrop, huj]
      simp
      have huw : u ++ [true] < t ++ [true] := by
      have huw' : u ++ [true] < (u ++ [true]).drop (j + 1) :=
      h.lt_drop (Nat.succ_pos _) (by simp using Nat.succ_lt_succ
hju)
      rw [htail] at huw'
      exact huw'
      rw [show (false :: (u ++ [true])).drop (j + 1) = false :: (t ++ [
true]) by
      simp [hdrop, huj]]
      exact cons_lt_cons huw
    | true =>
      rw [show (false :: (u ++ [true])).drop (j + 1) = true :: (t ++ [
true]) by
      simp [hdrop, huj]]
      exact (List.lt_iff_lex_lt _ _).2 <| List.Lex.rel (show false <
true by decide)

theorem prepend_false_preserves_isLyndon (u : List Bool)
  (h : IsSuffixLyndon (u ++ [true])) :
  IsLyndon (false :: (u ++ [true])) :=
  (prepend_false_preserves_isSuffixLyndon u h).toIsLyndon

theorem append_true_lt_true_cons (u : List Bool) :
  u ++ [true] < true :: (u ++ [true]) := by
  induction u with
  | nil =>
    exact (List.lt_iff_lex_lt _ _).2 <| List.Lex.cons List.Lex.nil
  | cons b u ih =>
    cases b with
    | false =>
      exact (List.lt_iff_lex_lt _ _).2 <| List.Lex.rel (show false < true by
decide)
    | true =>
      simp using cons_lt_cons (a := true) ih

```

```

theorem prepend_false_lt_self_append_true (u : List Bool) :
  false :: (u ++ [true]) < u ++ [true] := by
  induction u with
  | nil =>
    exact (List.lt_iff_lex_lt _ _).2 <| List.Lex.rel (show false < true by decide)
  | cons b u ih =>
    cases b with
    | false =>
      simpa using cons_lt_cons (a := false) ih
    | true =>
      exact (List.lt_iff_lex_lt _ _).2 <| List.Lex.rel (show false < true by
decide)

theorem replicate_falses_lt_self_append_true (k : Nat) (u : List Bool) :
  List.replicate (k + 1) false ++ (u ++ [true]) < u ++ [true] := by
  induction k with
  | zero =>
    simpa using prepend_false_lt_self_append_true u
  | succ k ih =>
    have h1 : false :: (List.replicate (k + 1) false ++ (u ++ [true])) < false
    :: (u ++ [true]) :=
      cons_lt_cons (a := false) ih
    have h2 : List.replicate (k + 2) false ++ (u ++ [true]) < false :: (u ++ [
true]) := by
      simpa [List.replicate, List.append_assoc] using h1
    exact lt_trans h2 (prepend_false_lt_self_append_true u)

theorem replicate_falses_lt_self_append_true_of_pos {i : Nat} (hi : 0 < i) (u :
  List Bool) :
  List.replicate i false ++ (u ++ [true]) < u ++ [true] := by
  cases i with
  | zero => cases Nat.lt_asymm hi hi
  | succ j =>
    simpa using replicate_falses_lt_self_append_true j u

theorem remove_first_true_preserves_isSuffixLyndon (k : Nat) (u : List Bool)
  (h : IsSuffixLyndon (List.replicate k false ++ (true :: (u ++ [true])))) :
  IsSuffixLyndon (List.replicate k false ++ (u ++ [true])) := by
  let t := u ++ [true]
  have hyx : List.replicate k false ++ t < List.replicate k false ++ (true :: t)
  := by
    simpa [t] using append_left_lt_of_lt (List.replicate k false) (
      append_true_lt_true_cons u)
  constructor
  · simp [t]
  · intro i hi0 hi
    by_cases hik : i ≤ k
    · have hdecomp : k = (k - i) + i := by omega
      have hy : List.replicate k false ++ t =
        List.replicate (k - i) false ++ (List.replicate i false ++ t) := by
        rw [hdecomp, List.replicate_add, List.append_assoc]
        simp
      have hdropy : (List.replicate k false ++ t).drop i = List.replicate (k - i)
      false ++ t := by
        rw [List.drop_append_of_le_length]
        · simp [hik, t]

```

```

· simp using hik
calc
  List.replicate k false ++ t
    = List.replicate (k - i) false ++ (List.replicate i false ++ t) := hy
_ < List.replicate (k - i) false ++ t :=
  append_left_lt_of_lt (List.replicate (k - i) false)
    (replicate_falses_lt_self_append_true_of_pos hi0 u)
_ = (List.replicate k false ++ t).drop i := hdropy.symm
· have hkl_t : k < i := lt_of_not_ge hik
  let j := i - k
  have hjpos : 0 < j := by
    dsimp [j]
    omega
  have hjlen : j < t.length := by
    dsimp [j] at *
    simp [t] at hi ⊢
    omega
  have hdropy : (List.replicate k false ++ t).drop i = t.drop j := by
    calc
      (List.replicate k false ++ t).drop i
        = ((List.replicate k false ++ t).drop k).drop j := by
            rw [show i = k + j by
              dsimp [j]
              omega, List.drop_drop]
        _ = t.drop j := by
            rw [List.drop_append_of_le_length]
            · simp [t]
            · simp
  have hdropx : (List.replicate k false ++ (true :: t)).drop (i + 1) = t.drop
j := by
  calc
    (List.replicate k false ++ (true :: t)).drop (i + 1)
      = ((List.replicate k false ++ (true :: t)).drop (k + 1)).drop j :=
by
    rw [show i + 1 = (k + 1) + j by
      dsimp [j]
      omega, List.drop_drop]
    _ = t.drop j := by
      have h_xk1 : (List.replicate k false ++ (true :: t)).drop (k + 1) = t
:= by
    calc
      (List.replicate k false ++ (true :: t)).drop (k + 1)
        = ((List.replicate k false ++ (true :: t)).drop k).drop 1 :=
by
    rw [show k + 1 = k + 1 by omega, List.drop_drop]
    _ = (true :: t).drop 1 := by
      rw [List.drop_append_of_le_length]
      · simp
      · simp
      _ = t := by simp
    rw [h_xk1]
  have hxlt : List.replicate k false ++ (true :: t) < t.drop j := by
  have hxlen : i + 1 < (List.replicate k false ++ (true :: t)).length := by
    simp [t] at hi ⊢
    omega
  have htmp := h.lt_drop (Nat.succ_pos _) hxlen
  rw [hdropx] at htmp

```

```

    exact htmp
  rw [hdropy]
  exact lt_trans hyx hxlt

/-- Suffix-form version of the main combinatorial transformation:
from '0~k 1 u 1', delete the first '1' after the initial zero block, then
prepend one more '0'. -/
theorem zero_block_step_isSuffixLyndon (k : Nat) (u : List Bool)
  (h : IsSuffixLyndon (List.replicate k false ++ (true :: (u ++ [true])))) :
  IsSuffixLyndon (List.replicate (k + 1) false ++ (u ++ [true])) := by
  have h' : IsSuffixLyndon ((List.replicate k false ++ u) ++ [true]) := by
    simp [List.append_assoc] using remove_first_true_preserves_isSuffixLyndon k
    u h
  simp [List.replicate, List.replicate_add, List.append_assoc] using
    prepend_false_preserves_isSuffixLyndon (List.replicate k false ++ u) h'

/-- Rotation-form corollary of 'zero_block_step_isSuffixLyndon'. -/
theorem zero_block_step_isLyndon (k : Nat) (u : List Bool)
  (h : IsSuffixLyndon (List.replicate k false ++ (true :: (u ++ [true])))) :
  IsLyndon (List.replicate (k + 1) false ++ (u ++ [true])) :=
  (zero_block_step_isSuffixLyndon k u h).toIsLyndon

theorem zeroes_then_one_isSuffixLyndon (k : Nat) :
  IsSuffixLyndon (List.replicate k false ++ [true]) := by
  induction k with
  | zero =>
    simp using isSuffixLyndon_singleton true
  | succ k ih =>
    simp [List.replicate, List.replicate_add] using
      prepend_false_preserves_isSuffixLyndon (List.replicate k false) ih

end A328596

```

A.3 Sequence.lean

```

import Mathlib.Data.Nat.PadicValNat
import Mathlib.Tactic
import A328596.BinaryWords
import A328596.Lyndon

/-!
# The Sequence A328596

This file packages the reversed-binary Lyndon condition into a predicate on
natural numbers and records a first OEIS sanity check.
-/

namespace A328596

/-- Membership in OEIS A328596:
'n' is positive and its reversed binary expansion is a Lyndon word. -/
def InA328596 (n : Nat) : Prop :=
  0 < n ∧ IsLyndon (revBinary n)

/-- A Boolean version of 'InA328596', used for computed examples. -/

```

```

def inA328596Bool (n : Nat) : Bool :=
  decide (0 < n) && isLyndonBool (revBinary n)

/-- A small computed prefix, used only as a sanity check against OEIS. -/
def firstTermsUpTo (n : Nat) : List Nat :=
  (List.range (n + 1)).filter inA328596Bool

example : firstTermsUpTo 32 = [1, 2, 4, 6, 8, 12, 14, 16, 20, 24, 26, 28, 30, 32]
:= by
  decide

theorem zeroes_then_one_isLyndon (k : Nat) :
  IsLyndon (List.replicate k false ++ [true]) := by
  exact (zeroes_then_one_isSuffixLyndon k).toIsLyndon

theorem two_pow_mem_a328596 (k : Nat) : InA328596 (2 ^ k) := by
  constructor
  · have hk : 0 < 2 ^ k := by
      positivity
    exact hk
  · rw [revBinary_pow_two]
    exact zeroes_then_one_isLyndon k

theorem clear_lsb_shape_mem_a328596 (k r : Nat) (hr : r ≠ 0)
  (hA : InA328596 (2 ^ k * (2 * r + 1))) :
  InA328596 (2 ^ (k + 1) * r) := by
  constructor
  · exact Nat.mul_pos (by positivity) (Nat.pos_of_ne_zero hr)
  · rcases revBinary_pos_ends_with_true (Nat.pos_of_ne_zero hr) with ⟨u, hu⟩
    have hword : IsSuffixLyndon (List.replicate k false ++ (true :: (u ++ [true])))
    := by
      have hly : IsLyndon (List.replicate k false ++ (true :: (u ++ [true]))) :=
        by
          simp [revBinary_pow_two_mul_odd, hu] using hA.2
          exact hly.toIsSuffixLyndon
      have hsmall : IsLyndon (List.replicate (k + 1) false ++ (u ++ [true])) :=
        zero_block_step_isLyndon k u hword
      simp [revBinary_mul_pow_two (k + 1) r hr, hu, List.append_assoc] using
        hsmall

theorem a328596_additive {n : Nat} (hn : 1 < n) (hA : InA328596 n) :
  ∃ a b, InA328596 a ∧ InA328596 b ∧ n = a + b := by
  let k := padicValNat 2 n
  let q := n.divMaxPow 2
  have hfact : 2 ^ k * q = n := by
    simp [k, q] using (pow_padicValNat_mul_divMaxPow 2 n)
  by_cases hq1 : q = 1
  · have hkshape : n = 2 ^ k := by
      simp [hq1] using hfact.symm
    rcases Nat.eq_zero_or_pos k with hk0 | hkpos
    · have hnone : n = 1 := by simp [hk0] using hkshape
      omega
    · obtain ⟨j, hj⟩ := Nat.exists_eq_succ_of_ne_zero (Nat.ne_of_gt hkpos)
      refine ⟨2 ^ j, 2 ^ j, two_pow_mem_a328596 j, two_pow_mem_a328596 j, ?_⟩
      calc
        n = 2 ^ (j + 1) := by simp [hj] using hkshape
        _ = 2 ^ j + 2 ^ j := by

```

```

    rw [Nat.pow_succ]
    nth_rw 2 [show 2 = 1 + 1 by rfl]
    rw [Nat.mul_add]
    simp
· have hqpos : 0 < q := by
  apply Nat.pos_of_ne_zero
  intro hq0
  rw [hq0, mul_zero] at hfact
  exact (Nat.ne_of_gt hA.1) hfact.symm
have hqodd : ¬ 2 ∣ q := by
  simp [q] using (Nat.not_dvd_divMaxPow (by decide : 1 < 2) (Nat.ne_of_gt hA
.1))
have hbodd : q.bodd = true := by
  cases hqbod : q.bodd <|> simp [Nat.mod_two_of_bodd, hqbod, Nat.
dvd_iff_mod_eq_zero] at hqodd ⊢
let r := q.div2
have hqshape : q = 2 * r + 1 := by
  have hbit : Nat.bit true r = q := by
    simp [r, hbodd] using (Nat.bit_bodd_div2 q)
  simp [Nat.bit, r] using hbit.symm
have hr : r ≠ 0 := by
  intro hr0
  have : q = 1 := by simp [hqshape, r, hr0]
  exact hq1 this
have hshape : n = 2 ^ k * (2 * r + 1) := by
  rw [← hfact, hqshape]
have hsmall : InA328596 (2 ^ (k + 1) * r) := by
  have hAshape : InA328596 (2 ^ k * (2 * r + 1)) := by
    simp [hshape] using hA
  exact clear_lsb_shape_mem_a328596 k r hr hAshape
refine ⟨2 ^ k, 2 ^ (k + 1) * r, two_pow_mem_a328596 k, hsmall, ?_⟩
calc
n = 2 ^ k * (2 * r + 1) := hshape
_ = 2 ^ k + 2 ^ (k + 1) * r := by
  rw [Nat.mul_add, Nat.mul_one, Nat.pow_succ]
  ac_rfl
end A328596

```

References

- [1] OEIS Foundation Inc., *The On-Line Encyclopedia of Integer Sequences*, Sequence A348369, <https://oeis.org/A348369>.
- [2] OEIS Foundation Inc., *The On-Line Encyclopedia of Integer Sequences*, Sequence A328596, <https://oeis.org/A328596>.
- [3] The Lean Community, *Lean 4 and Mathlib*, <https://leanprover-community.github.io/>.